CRM SERVER DEPLOYMENT RUNBOOK

Step-by-step production deployment guide | Frontend, Backend, Database, Nginx, PM2 & System Health

Environment reference

Item	Known configuration
Website	https://crm.tuktuk.live
EC2 public IP	3.109.90.44
Application directory	/home/ubuntu/crm-app-new
Frontend directory	/home/ubuntu/crm-app-new/dist
Backend entry point	/home/ubuntu/crm-app-new/server/src/index.js
PM2 process	crm-api
Backend local port	3000 (verify before use)
Database	yyjcpl_crm
S3 backup bucket	crm-tuktuk-backups-352306493926
AWS region	ap-south-1

Operating rule: Run one section at a time. Review outputs before moving on. These commands are a reusable guide, not a guarantee that every ZIP has the same structure.

Last prepared: 14 October 2026. Confirm paths, file names, current service configuration, and developer release notes before every production deployment.

1. Safety rules

- Never copy the ZIP's .env over the live /home/ubuntu/crm-app-new/server/.env.
- Never replace server/uploads with a release ZIP; user-uploaded files must be preserved.
- Do not run SQL files automatically. Inspect every migration and obtain approval first.
- Do not delete rollback folders, release ZIPs, or backups until deployment and user acceptance tests pass.
- Do not run broad recursive ownership/permission changes or delete directories with wildcards.
- For backend updates, preserve the existing localhost binding expected by Nginx unless the reviewed design explicitly requires a change.
- Commands below use the known production paths; verify current PM2/Nginx configuration before changing them.

2. Pre-deployment health checks

2.1 PM2

```
pm2 status
pm2 show crm-api
pm2 logs crm-api --lines 50
```

Confirm crm-api is online, its script path and working directory match production, and there are no new startup/database errors. Logs may contain sensitive data; redact secrets before sharing.

2.2 Nginx

```
sudo systemctl status nginx --no-pager
sudo nginx -t
```

Expected: Nginx active and syntax test successful. If nginx -t fails, do not reload or edit configuration blindly.

2.3 EC2 system health

```
uptime
free -h
df -h
top -b -n 1 | head -20
```

Check load, available memory, and disk free space. Stop if the root disk is nearly full or the host is unhealthy.

2.4 Public website and local API

```
curl -sS -o /dev/null -w 'Website HTTP %{http_code}\n' https://crm.tuktuk.live
sudo ss -ltnp | grep -E ':(80|443|3000)\\b' || true
```

A public HTTP 200 is a useful signal, not full functional testing. The local backend may return 401/404 on the root path; verify the actual health/API route if one exists. Port 3000 is based on the known setup and must be confirmed.

2.5 Current service configuration (read-only)

```
pm2 show crm-api
sudo nginx -T 2&gt;/dev/null | grep -nE 'server_name|proxy_pass|root ' | head -80
```

Use this to confirm the real document root and proxy target before deploying. Avoid sharing output that reveals private configuration.

3. Identify and upload the release ZIP

3.1 Upload from Windows PowerShell

```
scp -i "C:\\Users\\jaswa\\Downloads\\crm-server-key.pem" "C:\\Users\\jaswa\\Downloads\\&lt;ACTUAL-ZIP-NAME&gt;.zip" ubuntu@
```

Replace .zip with the exact file name. If the ZIP is elsewhere, use its real full path.

3.2 Verify and inspect on EC2

```
ls -lh /home/ubuntu/crm-release.zip
unzip -l /home/ubuntu/crm-release.zip | head -100
```

Do not deploy yet. Confirm the archive is the expected release and contains the expected frontend/backend/SQL files.

3.3 Extract to a fresh staging folder

```
RELEASE_DIR="/home/ubuntu/crm-release-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$RELEASE_DIR"
unzip -oq /home/ubuntu/crm-release.zip -d "$RELEASE_DIR"
echo "$RELEASE_DIR"
find "$RELEASE_DIR" -maxdepth 3 -type f | sort | head -150
```

Record the printed RELEASE_DIR and use that exact path for comparisons. Windows ZIPs may show backslash path warnings; verify actual extracted paths. Do not extract over the live application directory.

4. Classify what changed

4.1 Frontend asset references

```
grep -Eo 'src="[^"]+|href="[^"]+' /home/ubuntu/crm-app-new/dist/index.html
# Replace RELEASE_DIR below with the actual staging path:
grep -Eo 'src="[^"]+|href="[^"]+' "$RELEASE_DIR/dist/index.html"
```

4.2 Backend and dependency differences

```
diff -ru /home/ubuntu/crm-app-new/server/src "$RELEASE_DIR/server/src"
diff -u /home/ubuntu/crm-app-new/package.json "$RELEASE_DIR/package.json"
diff -u /home/ubuntu/crm-app-new/server/package.json "$RELEASE_DIR/server/package.json"
```

A diff exit status of 1 commonly means differences were found; it is not itself a deployment failure. A missing path means the release layout differs and must be investigated.

4.3 Find SQL migrations and environment files

```
find "$RELEASE_DIR" -type f \\( -iname '*.sql' -o -iname '*migration*' \\)
find "$RELEASE_DIR" -type f -name '.env*' -print
```

Never paste secret values into chat or logs. Treat unexpected .env files as sensitive; do not copy them to production. Review SQL contents before deciding whether a migration is needed.

5. Create/verify rollback points

5.1 Frontend snapshot

```
cd /home/ubuntu/crm-app-new
FRONTEND_BACKUP="/home/ubuntu/crm-app-new/dist-backup-$(date +%Y%m%d-%H%M%S)"
cp -a /home/ubuntu/crm-app-new/dist "$FRONTEND_BACKUP"
echo "$FRONTEND_BACKUP"
ls -ld "$FRONTEND_BACKUP"
```

Record the exact path. Do not remove older rollback folders.

5.2 Database backup

```
sudo /usr/local/bin/crm-db-backup.sh
aws s3 ls s3://crm-tuktuk-backups-352306493926/mysql/yyjcpl_crm/ --recursive --region ap-south-1 | tail -10
```

Confirm the script says the backup completed and the newly created S3 object is listed. This does not replace periodic restore testing.

5.3 Uploads backup check

```
sudo tail -n 20 /var/log/crm-uploads-backup.log
aws s3 ls s3://crm-tuktuk-backups-352306493926/uploads/ --region ap-south-1 | tail -10
```

The uploads job syncs the current uploads directory to S3. Confirm its latest log is successful. Application-code backup is manual by the user's preference; upload a sanitized release/rollback ZIP manually under the S3 application/ prefix if desired.

6. Frontend-only deployment

Use only when the release contains a new built dist/ and backend/API/database compatibility has been checked.

```
# Set this to the actual extracted release path:
RELEASE_DIR="/home/ubuntu/crm-release-YYYYMMDD-HHMMSS"
test -f "$RELEASE_DIR/dist/index.html" || { echo 'dist/index.html missing'; exit 1; }
# Ensure the backup in Section 5.1 exists before proceeding.
cp -a "$RELEASE_DIR/dist/." /home/ubuntu/crm-app-new/dist/
grep -Eo 'src="[^"]+|href="[^"]+' /home/ubuntu/crm-app-new/dist/index.html
```

Then open https://crm.tuktuk.live, hard-refresh with Ctrl+F5, and test login, dashboard, navigation, images, forms, buttons, and data operations. Frontend-only deployment usually does not require PM2 restart.

6.1 Frontend rollback

```
# Use the exact saved FRONTEND_BACKUP path from Section 5.1:
# First preserve the failed release's dist in a new folder, then restore:
FAILED_DIST="/home/ubuntu/crm-app-new/dist-failed-$(date +%Y%m%d-%H%M%S)"
sudo cp -a /home/ubuntu/crm-app-new/dist "$FAILED_DIST"
# Set FRONTEND_BACKUP to the exact saved path before running:
sudo cp -a "$FRONTEND_BACKUP/." /home/ubuntu/crm-app-new/dist/
```

Validate the restored site in a browser. Do not run this until FRONTEND_BACKUP is set to the correct verified directory.

7. Backend deployment

Backend updates are higher risk. Only proceed after reviewing code and dependency changes and confirming a rollback copy of the current backend and package/lock files. Do not use a generic whole-folder overwrite.

- Preserve /home/ubuntu/crm-app-new/server/.env and /home/ubuntu/crm-app-new/server/uploads.
- Review the actual PM2 script path, working directory, Node version, and existing listen binding. Current known entry point is server/src/index.js; known binding is 127.0.0.1 behind Nginx.
- If dependencies changed, review lockfile compatibility and use the correct package manager/lockfile. Do not run npm install blindly in production.
- Copy only reviewed changed files, preserving ownership and permissions. Keep the old versions in a dated rollback directory.

7.1 After reviewed backend changes

```
pm2 restart crm-api
pm2 status
pm2 logs crm-api --lines 100
curl -sS -o /dev/null -w 'Website HTTP %{http_code}\n' https://crm.tuktuk.live
sudo nginx -t
```

If startup fails, inspect logs and stop repeated restarts. Test actual API routes and critical workflows. Do not restart Nginx unless its configuration changed; if it changed, validate with nginx -t first, then use sudo systemctl reload nginx.

7.2 Backend rollback

Restore the exact prior backend source, package files and compatible dependency state from the recorded rollback directory. Restart crm-api and verify logs, API behavior, and website. Do not restore a database snapshot unless a separate approved database recovery plan calls for it.

8. Database/schema deployment

Do not run any SQL just because a ZIP contains it. Ask the developer what the migration changes and whether it is backward compatible.

- Inspect SQL for ALTER, DROP, DELETE, TRUNCATE, data rewrites, index changes, and assumptions about existing data.
- Take a fresh database backup and verify the new S3 object before any production migration.
- Prefer a staging/test database first. Plan downtime/maintenance if the change requires it.
- Apply only reviewed, approved statements using the team's established migration procedure. This runbook intentionally does not include a generic command to execute unknown SQL.
- Verify affected tables/columns/indexes and test the application against the changed schema.

Database restore is not the same as rolling back code. Restoring an older database can overwrite newer production data. Use a reviewed recovery plan and involve the database owner for destructive or irreversible changes.

9. Nginx and system health checks after deployment

```
pm2 status
sudo systemctl status nginx --no-pager
sudo nginx -t
uptime
free -h
df -h
sudo ss -ltnp | grep -E ':(80|443|3000)\\b' || true
curl -sS -o /dev/null -w 'Public website HTTP %{http_code}\n' https://crm.tuktuk.live
sudo tail -n 50 /var/log/nginx/error.log
```

Check PM2 is online, Nginx is active and valid, disk/memory are healthy, ports match expected configuration, and no new Nginx/backend errors appear. HTTP status alone is not sufficient.

Functional acceptance checklist

- Login/logout works.
- Dashboard and navigation pages load.
- Images and existing uploaded documents are present.
- Changed forms/buttons work and expected data saves correctly.
- Critical API endpoints respond as expected.
- No unexpected PM2 restart loop or new error spikes.
- Database and uploads backup paths remain intact.
- Developer or business owner confirms acceptance.

10. S3 backup reference

Existing automated jobs (confirmed from the server configuration):

```
Database: daily 02:00 UTC (07:30 IST)
  /etc/cron.d/crm-db-backup
  /usr/local/bin/crm-db-backup.sh
Uploads: daily 02:30 UTC (08:00 IST)
  /etc/cron.d/crm-uploads-backup
  /usr/local/bin/crm-uploads-backup.sh
```

Application code backup is not configured as an automatic job. The user prefers manual ZIP uploads. Store only sanitized code archives and never put live secrets in a ZIP.

```
aws s3 ls s3://crm-tuktuk-backups-352306493926/mysql/yyjcpl_crm/ --recursive --region ap-south-1 | tail -10
aws s3 ls s3://crm-tuktuk-backups-352306493926/uploads/ --region ap-south-1 | tail -10
```

Never assume a backup is valid solely because a scheduled log exists. Check S3 object presence and periodically test restores in a safe environment.

11. Final deployment record

Record	Fill in for every release
Release ZIP name / checksum	
Deployment date and operator	
Release type	Frontend / Backend / Database / Mixed
Frontend backup path	
DB backup S3 key	
Files/SQL changed	
PM2/Nginx/system checks	Pass / Fail
Browser/API acceptance	Pass / Fail
Rollback needed?	No / Yes — details:
Owner approval	

End of runbook. When a new ZIP arrives, inspect its contents and diffs first; use only the relevant deployment path, one step at a time.